

Système de gestion d'un restaurant

Institut Supérieur de l'Informatique et de la Modélisation Géologique de Gabès

RAPPORT DE MINI-PROJET

Programmation Orientée Objet — Java

Système de Gestion d'un Restaurant

Application Java Swing · Architecture MVC + DAO · Base de données MySQL

Module	Programmation Orientée Objet Java
Filière	Cycle Préparatoire Intégré — MPI/MI
Niveau	CPI 2 — Groupe A
Enseignant	M. Sofiane HACHICHA
Encadrante TP	Mme. Nesrine AKROUT
Année universitaire	2025 — 2026

Préparé par :

Ahmed Aoun
Issra Abdelouahed

Table des matières

1. Introduction	3
1.1 Contexte et objectifs	3
1.2 Technologies utilisées.....	3
2. Analyse des besoins	4
2.1 Acteurs du système	4
2.2 Fonctionnalités détaillées	4
2.2.1 Gestion des utilisateurs	4
2.2.2 Gestion des menus et plats	4
2.2.3 Gestion des commandes.....	4
2.2.4 Facturation	4
3. Conception UML	5
3.1 Diagramme de cas d'utilisation	5
3.2 Diagramme de classes.....	5
3.3 Relations entre les classes.....	5
4. Conception de la base de données	8
4.1 Schéma relationnel.....	8
4.2 Structure détaillée des tables.....	8
Table : utilisateur	8
Table : commande	8
4.3 Script SQL	10
5. Architecture MVC + DAO	11
5.1 Description de l'architecture	11
5.2 Patron DAO — Interface générique	11
5.3 Structure des packages Java	11
6. Interfaces graphiques.....	12
6.1 Interface de connexion (LoginFrame).....	12
6.2 Interface de gestion des menus (GestionMenuFrame).....	12
6.3 Tableau de bord par rôle	12
6.3.1 Tableau de bord Client.....	12
6.3.2 Tableau de bord Serveuse.....	12
6.3.3 Tableau de bord Cuisinier	12
7. Conclusion	13

1. Introduction

Ce rapport présente la conception et le développement d'un système de gestion d'un restaurant réalisé dans le cadre du module de Programmation Orientée Objet (POO) du Cycle Préparatoire Intégré (CPI2) de l'ISIMG Gabès.

1.1 Contexte et objectifs

L'objectif principal du projet est de développer une application Java complète permettant la gestion des menus, des plats et des commandes d'un restaurant. L'application devra gérer trois types d'acteurs : les clients, les serveuses et les cuisiniers.

1.2 Technologies utilisées

- Langage de programmation : Java (JDK 24)
- Interface graphique : Java Swing
- Base de données : MySQL 8.0
- Connecteur JDBC : MySQL Connector/J 9.5
- Architecture : MVC (Modèle-Vue-Contrôleur) + Patron DAO
- Outil de développement : VS Code / Eclipse

2. Analyse des besoins

2.1 Acteurs du système

Le système identifie trois types d'utilisateurs, chacun disposant d'un espace personnalisé après authentification.

Acteur	Fonctionnalités principales
Client	Parcourir le menu · Commander des plats
Serveuse	Parcourir/Commander · Voir commandes en cours et reçues · Générer une facture
Cuisinier	CRUD plats · Traiter/annuler commandes · Notifier la serveuse

2.2 Fonctionnalités détaillées

2.2.1 Gestion des utilisateurs

- Connexion sécurisée par nom d'utilisateur et mot de passe
- Affichage des informations personnelles après connexion
- Redirection vers l'espace dédié au rôle de l'utilisateur

2.2.2 Gestion des menus et plats

- Consultation du menu par catégorie (entrées, plats, desserts, boissons)
- Ajout, modification et suppression de plats (cuisinier uniquement)
- Gestion de la disponibilité des plats

2.2.3 Gestion des commandes

- Création d'une commande avec sélection de plusieurs plats et quantités
- Suivi du statut : EN_ATTENTE → EN_COURS → PRÊTE → SERVIE
- Annulation possible par le cuisinier avant ou pendant le traitement
- Notification automatique à la serveuse quand une commande est prête

2.2.4 Facturation

- Génération automatique de la facture à partir des lignes de commande
- Affichage du détail des plats consommés avec leurs prix
- Calcul du montant total et possibilité d'appliquer une remise

3. Conception UML

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous illustre les interactions entre les trois acteurs principaux du système et les fonctionnalités disponibles.

[Voir diagramme Use Case dans figure 1]

Les principaux cas d'utilisation identifiés sont :

- Se connecter au système (commun à tous les acteurs)
- Parcourir le menu (Client, Serveuse)
- Commander des plats (Client, Serveuse)
- Visualiser les commandes en cours et reçues (Serveuse)
- Générer une facture (Serveuse)
- Gérer les plats du menu — CRUD (Cuisinier)
- Traiter ou annuler une commande (Cuisinier)
- Notifier la serveuse lorsqu'une commande est prête (Cuisinier)

3.2 Diagramme de classes

Le diagramme de classes présente la structure objet du système avec les relations d'héritage, d'association et de composition. *[Voir diagramme Use Case dans figure 2]*

Classe	Attributs principaux	Méthodes principales
Utilisateur (abstraite)	id, nom, prenom, username, password, role	login(), getInfos()
Client	numTable	parcourirMenu(), commander()
Serveuse	numBadge	voirCommandes(), genererFacture()
Cuisinier	specialite	ajouterPlat(), traiterCommande(), notifier()
Menu	id, nom, description	getPlats(), addPlat()
Plat	id, nom, prix, disponible, idMenu	getInfo(), setDisponible()
Commande	id, date, statut, numTable	calculerTotal(), changerStatut()
LigneCommande	id, quantite, prixUnitaire	getSousTotal()
Facture	id, montantTotal, remise, montantFinal	generer(), afficher()

3.3 Relations entre les classes

- Utilisateur ◀|— Client, Serveuse, Cuisinier (héritage)
- Menu 1 —> 1..* Plat (composition : un menu contient des plats)
- Commande 1 —> 1..* LigneCommande (composition)
- LigneCommande 1 —> 1 Plat (association)
- Commande 1 —> 0..1 Facture (association)
- Client 1 —> 0..* Commande (association)
- Serveuse 1 —> 0..* Commande (association)

Diagramme de Cas d'Utilisation — Système de Gestion d'un Restaurant

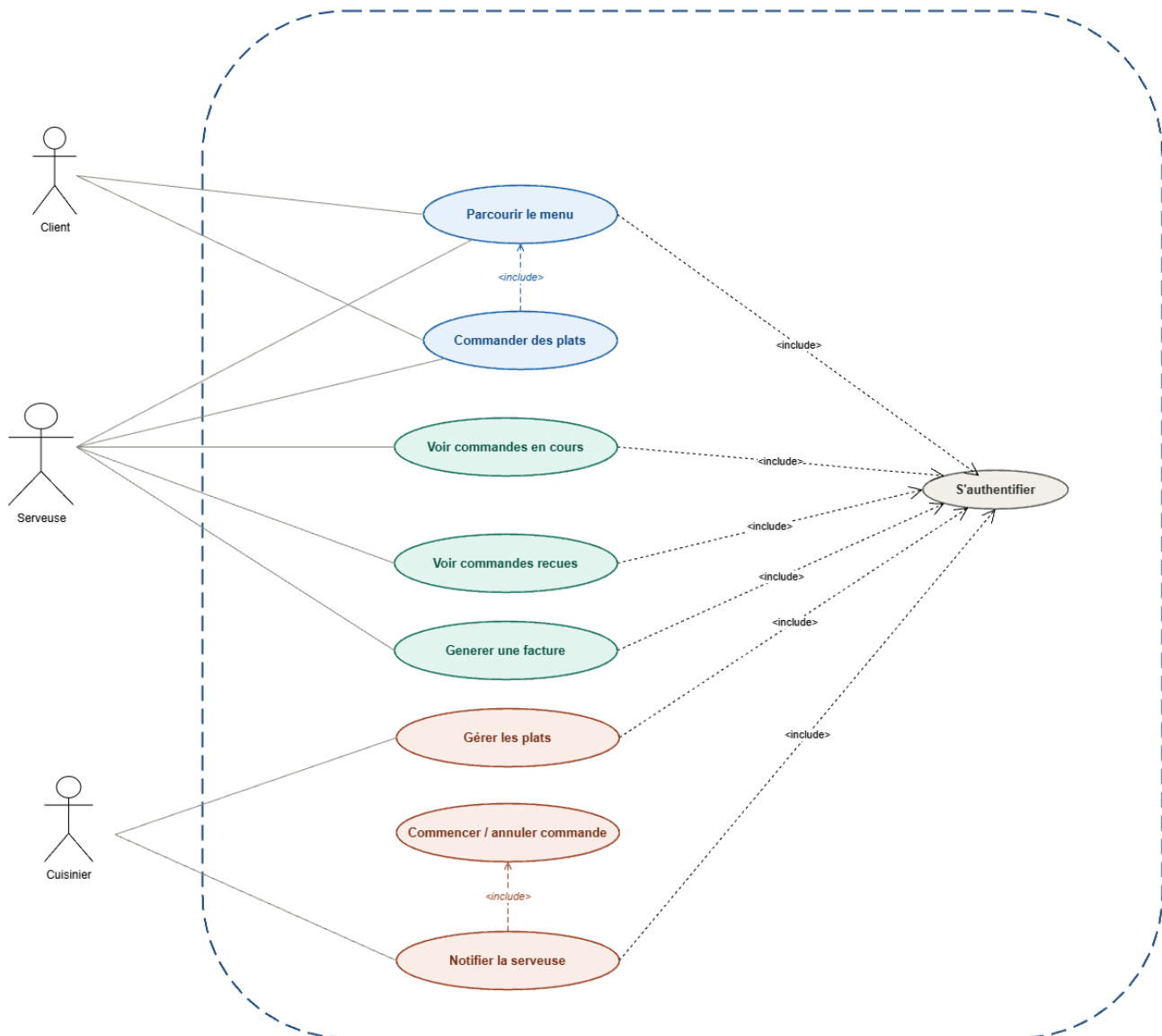


Figure 1 : Diagramme de cas d'utilisation

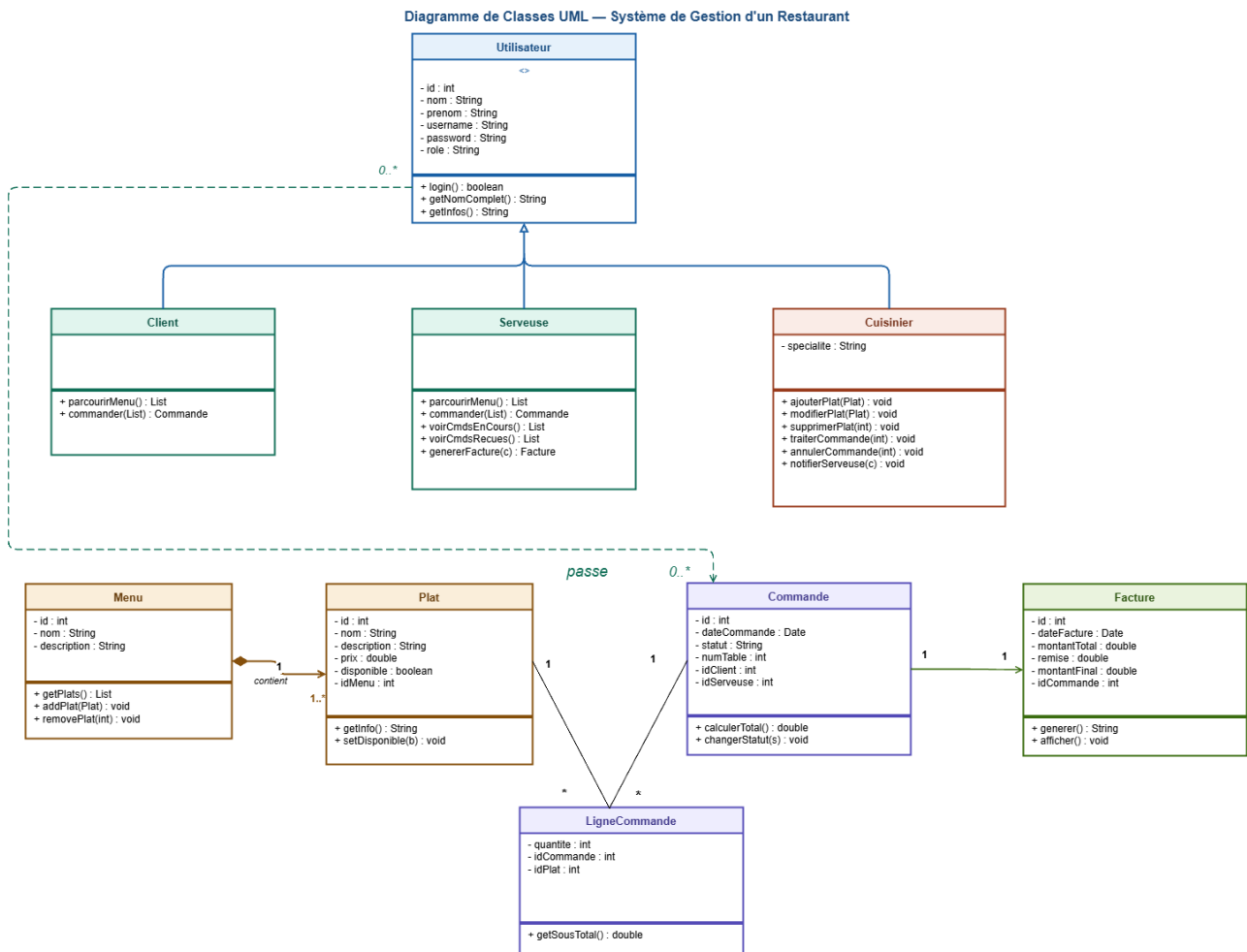


Figure 2 : Diagramme des classes

4. Conception de la base de données

4.1 Schéma relationnel

La base de données relationnelle restaurant_db est composée des tables suivantes :

Table	Description	Clé(s) étrangère(s)
utilisateur	Stocke les 3 types d'utilisateurs avec leur rôle	—
menu	Catégories du menu (Entrées, Plats, Desserts, Boissons)	—
plat	Plats disponibles avec prix et disponibilité	id_menu → menu(id)
commande	Commandes passées avec statut et date	id_client, id_serveuse → utilisateur(id)
ligne_commande	Lignes de détail d'une commande (plats + quantités)	id_commande → commande(id), id_plat → plat(id)
facture	Facture générée à partir d'une commande servie	id_commande → commande(id)
notification	Notifications cuisinier → serveuse lors d'une commande prête	id_commande, id_serveuse

4.2 Structure détaillée des tables

Table : utilisateur

Attribut	Type	Description
id	INT (PK)	Identifiant auto-incrémenté
nom	VARCHAR(50)	Nom de famille
prenom	VARCHAR(50)	Prénom
username	VARCHAR(30)	Identifiant de connexion (unique)
password	VARCHAR(255)	Mot de passe hashé (SHA-256)
role	ENUM	CLIENT SERVEUSE CUISINIER
actif	BOOLEAN	Indique si le compte est actif

Table : commande

Attribut	Type	Description
id	INT (PK)	Identifiant auto-incrémenté
date_commande	DATETIME	Date et heure de la commande
statut	ENUM	EN_ATTENTE EN_COURS PRETE SERVIE ANNULEE
num_table	INT	Numéro de la table du client
id_client	INT (FK)	Référence vers utilisateur (client)

Attribut	Type	Description
id_serveuse	INT (FK)	Référence vers utilisateur (serveuse)

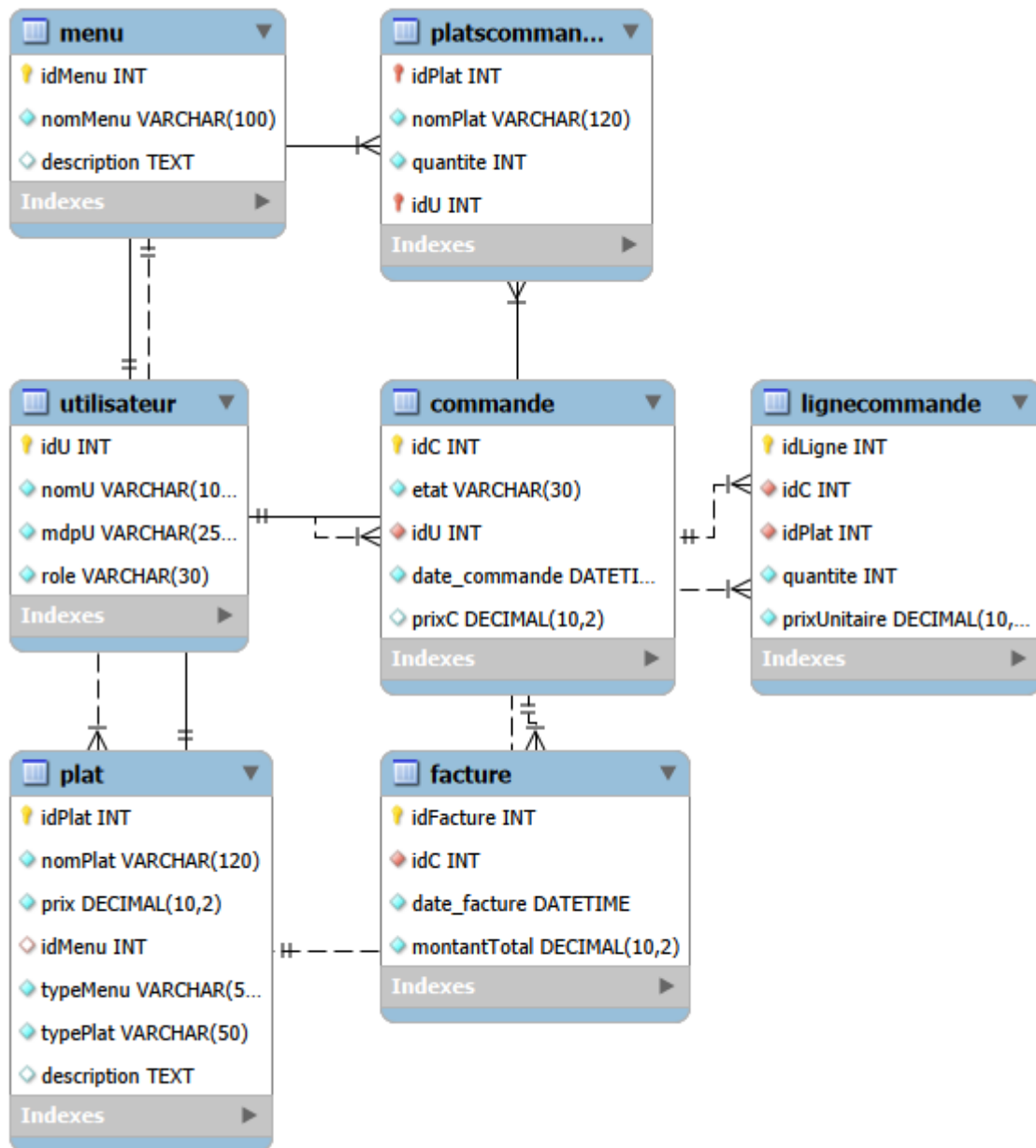


Figure 3 : Schéma relationnel

4.3 Script SQL

```

1  CREATE DATABASE IF NOT EXISTS mydatabase
2  CHARACTER SET utf8mb4
3  COLLATE utf8mb4_unicode_ci;
4
5  USE mydatabase;
6
7  CREATE TABLE IF NOT EXISTS menu (
8  idMenu INT PRIMARY KEY AUTO_INCREMENT,
9  nomMenu VARCHAR(100) NOT NULL,
10 description TEXT
11 );
12
13 CREATE TABLE IF NOT EXISTS utilisateur (
14 idU INT auto_increment PRIMARY KEY,
15 nomU VARCHAR(100) NOT NULL,
16 mdpU VARCHAR(255) NOT NULL,
17 role VARCHAR(30) NOT NULL
18 );
19
20 CREATE TABLE IF NOT EXISTS plat (
21 idPlat INT PRIMARY KEY AUTO_INCREMENT,
22 nomPlat VARCHAR(120) NOT NULL,
23 prix DECIMAL(10,2) NOT NULL,
24 idMenu INT NULL,
25 typeMenu VARCHAR(50) NOT NULL,
26 typePlat VARCHAR(50) NOT NULL,
27 description TEXT,
28 CONSTRAINT fk_plat_menu
29 FOREIGN KEY (idMenu) REFERENCES menu(idMenu)
30 ON DELETE SET NULL
31 ON UPDATE CASCADE
32 );
33
34 CREATE TABLE IF NOT EXISTS commande (
35 idC INT PRIMARY KEY AUTO_INCREMENT,
36 etat VARCHAR(30) NOT NULL,
37 idU INT NOT NULL,
38 date_commande DATETIME NOT NULL,
39 prixC DECIMAL(10,2) DEFAULT 0,
40 CONSTRAINT fk_commande_utilisateur
41 FOREIGN KEY (idU) REFERENCES utilisateur(idU)
42 ON DELETE CASCADE
43 ON UPDATE CASCADE
44 );
45
46 CREATE TABLE IF NOT EXISTS facture (
47 idFacture INT PRIMARY KEY AUTO_INCREMENT,
48 idC INT NOT NULL,
49 date_facture DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
50 montantTotal DECIMAL(10,2) NOT NULL DEFAULT 0,
51 CONSTRAINT fk_facture_commande
52 FOREIGN KEY (idC) REFERENCES commande(idC)
53 ON DELETE CASCADE
54 ON UPDATE CASCADE
55 );
56
57 CREATE TABLE IF NOT EXISTS lignecommande (
58 idLigne INT PRIMARY KEY AUTO_INCREMENT,
59 idC INT NOT NULL,
60 idPlat INT NOT NULL,
61 quantite INT NOT NULL,
62 prixUnitaire DECIMAL(10,2) NOT NULL,
63 CONSTRAINT fk_lignecommande_commande
64 FOREIGN KEY (idC) REFERENCES commande(idC)
65 ON DELETE CASCADE
66 ON UPDATE CASCADE,
67 CONSTRAINT fk_lignecommande_plat
68 FOREIGN KEY (idPlat) REFERENCES plat(idPlat)
69 ON DELETE CASCADE
70 ON UPDATE CASCADE,
71 CONSTRAINT chk_lignecommande_quantite_positive
72 CHECK (quantite > 0)
73 );
74
75 CREATE TABLE IF NOT EXISTS platscommande (
76 idPlat INT NOT NULL auto_increment,
77 nomPlat VARCHAR(120) NOT NULL,
78 quantite INT NOT NULL,
79 idU INT NOT NULL,
80 PRIMARY KEY (idPlat, idU),
81 CONSTRAINT fk_platscommande_plat
82 FOREIGN KEY (idPlat) REFERENCES plat(idPlat)
83 ON DELETE CASCADE
84 ON UPDATE CASCADE,
85 CONSTRAINT fk_platscommande_utilisateur
86 FOREIGN KEY (idU) REFERENCES utilisateur(idU)
87 ON DELETE CASCADE
88 ON UPDATE CASCADE,
89 CONSTRAINT chk_quantite_positive
90 CHECK (quantite > 0)
91 );

```

5. Architecture MVC + DAO

5.1 Description de l'architecture

L'application respecte une architecture en trois couches séparées, conformément au patron MVC combiné avec le patron DAO pour l'accès aux données.

Couche	Package Java	Rôle
Modèle (Model)	model/	Classes Java représentant les entités métier (Plat, Commande, Utilisateur...)
Vue (View)	view/	Interfaces graphiques Java Swing (JFrame, JDialog, JPanel...)
Contrôleur (Controller)	controller/	Logique métier, coordination entre la vue et le DAO
Accès données (DAO)	dao/	Requêtes SQL JDBC (insert, update, delete, findAll, findById)
Utilitaires	util/	Connexion à la BD (Singleton DBConnection)

5.2 Patron DAO — Interface générique

Une interface générique `IDao<T>` définit les opérations CRUD communes à toutes les entités :

```

boolean insert(T obj)  — Insertion d'un objet dans la base de données
boolean update(T obj)  — Mise à jour d'un objet existant
boolean delete(int id) — Suppression par identifiant
T findById(int id)     — Recherche d'un enregistrement par son ID
List<T> findAll()      — Récupération de tous les enregistrements

```

5.3 Structure des packages Java

- model/ → Utilisateur (abstraite), Client, Serveuse, Cuisinier, Menu, Plat, Commande, LigneCommande, Facture
- dao/ → `IDao<T>`, UtilisateurDAO, MenuDAO, PlatDAO, CommandeDAO, LigneCommandeDAO, FactureDAO
- controller/ → AuthController, MenuController, CommandeController, FactureController
- view/ → LoginFrame, DashboardClientFrame, DashboardServeuseFrame, DashboardCuisinierFrame, GestionMenuFrame, CommandeFrame
- util/ → DBConnection (Singleton)

6. Interfaces graphiques

6.1 Interface de connexion (LoginFrame)

La fenêtre de connexion est la première interface présentée à l'utilisateur. Elle comprend :

- Un champ de saisie pour le nom d'utilisateur
- Un champ de saisie pour le mot de passe (masqué)
- Un bouton « Se connecter » qui vérifie les identifiants en base de données
- Un message d'erreur en cas d'identifiants incorrects
- Une redirection automatique vers le tableau de bord selon le rôle

6.2 Interface de gestion des menus (GestionMenuFrame)

Cette interface est accessible au cuisinier pour la gestion complète des plats. Conformément à l'énoncé, elle contient :

Bouton	Fonctionnalité
<< Ajouter >>	Ouvre un formulaire de dialogue pour insérer un nouveau plat dans la liste
<< Modifier >>	Permet de modifier les données du plat sélectionné dans la table
<< Supprimer >>	Supprime le plat sélectionné après confirmation de l'utilisateur
<< Enregistrer >>	Sauvegarde définitivement les modifications dans la base de données MySQL
<< Fermer >>	Ferme la fenêtre de gestion des menus

6.3 Tableau de bord par rôle

6.3.1 Tableau de bord Client

- Accès au menu pour consultation des plats disponibles
- Formulaire de commande pour sélectionner des plats et valider une commande

6.3.2 Tableau de bord Serveuse

- Vue des commandes en cours de traitement
- Vue des commandes reçues et en attente
- Génération de facture pour les commandes servies

6.3.3 Tableau de bord Cuisinier

- Interface CRUD des plats (accès à GestionMenuFrame)
- Liste des commandes demandées avec actions : Commencer / Annuler
- Liste des commandes en cours avec action : Marquer comme Prête
- Notification automatique à la serveuse lors du passage à l'état PRÊTE

7. Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la Programmation Orientée Objet en Java à travers un cas concret et complet. L'implémentation d'une architecture MVC combinée au patron DAO assure une séparation claire entre les couches de présentation, de logique métier et d'accès aux données.

Les principaux apports de ce projet sont :

- Maîtrise de l'héritage et du polymorphisme à travers la hiérarchie Utilisateur → Client/Serveuse/Cuisinier
- Conception et implémentation d'une interface générique DAO<T> réutilisable
- Développement d'interfaces graphiques ergonomiques avec Java Swing
- Connexion et manipulation d'une base de données MySQL via JDBC
- Application du patron de conception Singleton pour la gestion de la connexion

Ce système peut être enrichi par des fonctionnalités avancées telles que la génération de factures en PDF, un système de réservation en ligne, des statistiques de vente ou l'intégration d'une API REST.